

Embedding Functions. We will use d to denote the dimension of the embedding and w_i to denote the i -th coordinate vector in $\mathbb{R}^d$.

We will separate the embedding function into two parts: the word embedding and the position embedding. For the word embedding, we will use $iw_1 \in \mathbb{R}^d$ to represent the embedding of the vertice i in the tokenization of the graph. For the k -th special token, we will use w_{2+k} to represent its embedding. For example, the embedding of $\sim$ is w_2 . We will denote the word embedding matrix as $W^{(E)} \in \mathbb{R}^{|V| \times d}$.

For the position embedding, we will use lw_d to represent the position embedding of the l -th token in the tokenization of the graph, which is a hyperparameter. The final embedding of any token sequence is the sum of the word embedding and the position embedding. We will use Emb to denote the embedding function.

This embedding function will be fixed and shared across all models we consider in this paper and will not be learned during training, hence we will not consider it as part of the model parameters.

Language Modeling. In this work, we will consider the difference between Transformer and Recurrent Neural Networks on reasoning tasks, which is a special case of language modeling. We will define language modeling as follows:

Definition A.3 (Language Model). A language model is a function $M : \cup_{l=1}^L |V|^l \rightarrow \mathbb{P}_{|V|}$, where $|V|$ is the vocabulary size, l is the sequence length, and $\mathbb{P}_{|V|}$ is the probability simplex over $|V|$.

A.4 Language Models for Reasoning.

Chain of Thought. We will now define how we use language models to solve reasoning tasks utilizing the following technique called chain of thought.

Definition A.4 (Chain of Thought). Given a language model, M with vocabulary V and the tokenized input sequence $\mathcal{S}_{\text{in}} \in |V|^{l_0}$, chain of thought (CoT) generates the following sequence of tokenized sequence:

$$\mathcal{S}_0 = \mathcal{S}_{\text{in}}, \quad (7)$$

$$s_i^{\text{next}} = \arg \max_{j \in V} M(\mathcal{S}_i)[j], \quad (8)$$

$$\mathcal{S}_{i+1} = \mathcal{S}_i \oplus s_i^{\text{next}}, \forall i \geq 0. \quad (9)$$

The process terminates at $\mathcal{S}_i$ when $\arg \max_{j \in V} M(\mathcal{S}_i)[j]$ is YES or NO. The language model can solve the reasoning task within T steps of CoT if the process terminates at $\mathcal{S}_i$ where $i \leq T$ and the final output is correct. We will call the special case where the language model solves the reasoning task within 0 steps of CoT as solving the reasoning task without CoT.

Retrieval Augmentation. We will show in this paper that retrieval augmentation is a necessary technique to solve reasoning tasks for recurrent neural networks. We will define retrieval augmentation as follows:

Definition A.5 (Retrieval Augmented Generation). Retrieval Augmented Generation means giving the language model the capability to retrieve relevant information to assist generation. We formally described the process here. Given a language model M with vocabulary V (containing two additional special tokens called $\langle \text{StartSearch} \rangle$ and $\langle \text{EndSearch} \rangle$) and the tokenized input sequence $\mathcal{S}_{\text{in}} \in |V|^{l_0}$, retrieval augmented generation generates following sequence of tokenized sequence:

$$\begin{aligned} \mathcal{S}_0 &= \mathcal{S}_{\text{in}}, \\ s_i^{\text{next}} &= \arg \max_{j \in V} M(\mathcal{S}_i)[j], \\ \mathcal{S}_{i+1} &= \begin{cases} \mathcal{S}_i \oplus s_i^{\text{next}}, & s_i^{\text{next}} \neq \langle \text{EndSearch} \rangle \\ \mathcal{S}_i \oplus s_i^{\text{next}} \oplus \text{RETRIEVE}(\mathcal{S}_i), & \text{otherwise.} \end{cases} \end{aligned}$$

Here RETRIEVE looks for the last occurrence of $\langle \text{StartSearch} \rangle$ at position l_s and $\langle \text{EndSearch} \rangle$ in $\mathcal{S}$ at position l_e and treat $\text{Detokenize}(\mathcal{S}_{l_s:l_e})$ as a regular expression. The algorithm then uses the regular expression on $\text{Detokenize}(\mathcal{S}_{1:l_s-1})$. If the regular expression ever matches, the RETRIEVE will return the match. If the regular expression never matches, RETRIEVE will return a special token FAILED.

Similar to Definition A.4, we can define the notion of solving the reasoning task within T steps of retrieval augmented generation and solving the reasoning task without retrieval augmented generation.

We will note that assuming $|V| = O(n)$ and every search query and the result is of length $O(1)$, the regular expression evaluation can typically be evaluated in $O(n)$ time.

B Omitted Experiment Details

We train three different architectures: (1) LLaMA architecture (Touvron et al., 2023) representing Transformers, (2) Mamba architecture (Gu & Dao, 2023) representing RNNs, and (3) Mamba with one additional layer of LLaMA block representing hybrid architectures. Following our theory, we freeze and weight-tie the prediction head and word embedding in all the models. For ease of training, we use a different embedding function mapping i -th token to $[\sin(\frac{i}{10000^{j/d}}), \cos(\frac{i}{10000^{j/d}})]_{j \in [d/2]}$ with N being the number of different tokens and use standard RoPE (Su et al., 2024) as position embedding. We train every model with at least 1M samples to guarantee convergence using Adam with a cosine learning rate. If the model doesn't converge, we retrain using 5M samples. After a grid search over learning rates, we train all the Transformer models with learning rates $1e-3$ and the rest of the models with learning rates $3e-4$. We run all the experiments on a server with 8 A100s and the estimated time to reproduce the results is within 2 days.

C Omitted Proof

C.1 Building Blocks of FFNs Construction

We will first show some basic operations that multiple layers of feedforward neural networks with ReGLU activation can perform that will be used in the following proofs.

Lemma C.1 (Multiplication). *Given two dimensions i_1, i_2 , there exists a parameter configuration of a 1-layer feedforward neural network with ReGLU activation that for any input $x \in \mathbb{R}^d$ and constant width, computes the product of x_{i_1} and x_{i_2} .*

$$g(x) = [x_{i_1} \times x_{i_2}, 0, \dots, 0]^\top.$$

Proof. We can construct the following feedforward neural network with ReGLU activation:

$$\begin{aligned} W_1 x &= [x_{i_1} \quad -x_{i_1} \quad 0 \dots 0]^\top, \\ W_2 x &= [x_{i_2} \quad x_{i_2} \quad 0 \dots 0]^\top, \\ W_3 h &= [h_1 + h_2 \quad 0 \quad 0 \quad 0 \dots 0]^\top, \\ b_1 &= b_2 = b_3 = 0. \\ g(x) &= W_3 (\text{ReLU}(W_1 x) \otimes \text{ReLU}(W_2 x)) = [x_{i_1} \times x_{i_2}, 0, \dots, 0]^\top. \end{aligned}$$

□

Lemma C.2 (Linear Operations). *Given a linear transformation $W \in \mathbb{R}^{d \times d}$, there exists a parameter configuration of a 1-layer feedforward neural network with ReGLU activation and width $w = d$ that for any input $x \in \mathbb{R}^d$, computes Wx .*

Proof.

$$\begin{aligned} b_1 &= 1^w, b_2 = 0, b_3 = 0, \\ W_1 &= 0, W_2 = W, W_3 = I_{d \times d} \end{aligned}$$

□

Lemma C.3 (Indicator). *Given a constant integer $B \leq d$ and a dimension i , there exists a parameter configuration of a 1-layer feedforward neural network with ReGLU activation and width 4 that for any input $x \in \mathbb{R}^d$, computes the indicator function of $x_i = B$ when x_i is an integer.*

Proof.

$$\begin{aligned} b_2 &= 1^w, b_1 = [-B - 0.5, -B + 0.5, B - 0.6, B + 0.4]^\top, b_3 = 10, \\ W_2 &= 0, W_1 x = [x_i, x_i, -x_i, -x_i]^\top, \\ \text{ReLU}(W_1 x + b_1) &= [\text{ReLU}(x_i - B - 0.5) \quad \text{ReLU}(x_i - B + 0.5) \quad \text{ReLU}(B - x_i - 0.6) \quad \text{ReLU}(B - x_i + 0.4)] \\ W_3 &= 10 [1 \quad -1 \quad -1 \quad 1]^\top. \end{aligned}$$

Then,

$$\begin{aligned}
g(x) &= W_3 \text{ReLU}(W_1 x + b_1) + b_3 \\
&= 10 (\text{ReLU}(x_i - B - 0.5) - \text{ReLU}(x_i - B + 0.5)) + 10 (\text{ReLU}(B - x_i - 0.6) - \text{ReLU}(B - x_i + 0.4)) + 10 \\
&= \begin{cases} 10 \times 0 + 10 \times -1 + 10 = 0, & \text{if } x_i < B, \\ 10 \times -0.5 + 10 \times -0.4 + 10 = 1, & \text{if } x_i = B, \\ 10 \times -1 + 10 \times 0 + 10 = 0, & \text{if } x_i > B. \end{cases}
\end{aligned}$$

□

Lemma C.4 (Lookup Table). *For constant B and k such that $kB \leq d$, given a lookup table which key is tuple of k integers bounded by B , and value is a scalar in $\mathbb{R}_p$, there exists a parameter configuration of a 1-layer feedforward neural network with ReGLU activation with width $O(B^k)$ that for any input $x \in \mathbb{R}^d$, computes the value of the lookup table at the key $x_{i_1}, x_{i_2}, \dots, x_{i_k}$.*

Proof. We can calculate $x_{i_1} + B \times x_{i_2} + B^2 \times x_{i_3} + \dots + B^{k-1} \times x_{i_k}$, and then scale B^k indicator functions to get the value of the lookup table at the key $x_{i_1}, x_{i_2}, \dots, x_{i_k}$. □

Lemma C.5 (Threshold). *Given any threshold u and constant $\epsilon > 0$, there exists a parameter configuration of a 1-layer feedforward neural network with ReGLU activation and width 2 that for any input $x \in \mathbb{R}^d$, computes the indicator function $x_i > u$ on $x_i \in [-\infty, u - \epsilon] \cup [u, \infty]$.*

Proof.

$$\begin{aligned}
b_2 &= 1^w, b_1 = [-u + \epsilon, -u + 0.5\epsilon]^\top, b_3 = 0, \\
W_2 &= 0, W_1 x = [x_i, x_i]^\top, \\
\text{ReLU}(W_1 x + b_1) &= [\text{ReLU}(x_i - u + \epsilon) \quad \text{ReLU}(x_i - u + 0.5\epsilon)] \\
W_3 &= \frac{2}{\epsilon} [1 \quad -1 \quad -1 \quad 1]^\top.
\end{aligned}$$

Then,

$$\begin{aligned}
g(x) &= W_3 \text{ReLU}(W_1 x + b_1) + b_3 \\
&= 2/\epsilon (\text{ReLU}(x_i - u - \epsilon) - \text{ReLU}(x_i - u + 0.5)) \\
&= \begin{cases} 0 & \text{if } x_i < u - \epsilon, \\ 1, & \text{if } x_i > u, \end{cases}
\end{aligned}$$

□

Lemma C.6 (Interval). *Given any constant u, v and $\epsilon > 0$, there exists a parameter configuration of a 1-layer feedforward neural network with ReGLU activation and width 4 that for any input $x \in \mathbb{R}^d$, computes the indicator function $x_i \in [u, v]$ on $x_i \in [-\infty, u - \epsilon] \cup [u, v] \cup [v, \infty]$.*

Proof. The interval function here can be written as the difference of two threshold functions. We can use the same construction as in Lemma C.5 to approximate the indicator function $x_i > u$ and $x_i > v + \epsilon$ and then take the difference. □

C.2 Building Blocks of Transformers Construction

We will show in this section some construction for basic functionality using Transformer Blocks. This construction will be used in the following sections to prove the main results.

We will always use $X \in \mathbb{R}_p^{d \times l}$ as the input to the Transformer Block, where d is the dimension of the input, and l is the length of the sequence. We will first outline all the building functionality and then show how to implement them.

Definition C.7 (Copying Function). For integer s , index set $I_1, I_2 \subset [d - 20]$ satisfying $|I_1| = |I_2|$, a copying function $\text{COPY}[s, I_1, I_2]$ satisfies the following, $\forall X \in \mathbb{R}_p^{d \times l}$, then

$$\begin{aligned}
\text{COPY}[s, I_1, I_2](X)_{I_2, k} &= x_{I_1, \max\{k-s, 0\}} \quad \forall k \leq [m] \\
\text{COPY}[s, I_1, I_2](X)_{I_2^c, k} &= 0 \quad \forall r \in [m]
\end{aligned}$$